# ReqShift and GoalShift

## What they are, how they work, and how they might become a product

Daniyal Bukhari

August 21, 2026

**The one sentence version.** A language model cannot tell you when it is wrong. A SAT solver cannot read prose. Their weaknesses are opposites, so **ReqShift** uses AI to turn a requirements document into a formal model, and **GoalShift** then proves what follows from it. Nothing the tool *claims* comes from the AI.

This document has two halves. Sections 1 to 5 describe the software as it stands today, with real measured numbers. Sections 6 to 9 are about turning it into something sellable, and those are ideas and options rather than facts. Where something is uncertain or unverified, it says so.

# Contents

**1 The short version** **3**

**2 How GoalShift works** **3**
2.1 What goes in . . . 3
2.2 The four steps . . . 3
2.3 The scalability fix . . . 4
2.4 The reference numbers . . . 4

**3 How ReqShift works** **4**
3.1 The pipeline . . . 4
3.2 The files . . . 5
3.3 Why the AI cannot invent a defect . . . 5
3.4 What it finds . . . 5
3.5 Two ways to reach the AI . . . 5

**4 The SMT extension** **6**
4.1 Integer counts . . . 6
4.2 Timing and capacity . . . 6
4.3 How it is kept honest . . . 6

**5 Evidence that it works** **7**
5.1 It finds real defects . . . 7
5.2 It reproduces a model you built by hand . . . 7
5.3 It reads numbers out of prose . . . 7

**6 From prototype to product: the honest starting position** **8**
6.1 What you have . . . 8
6.2 What you do not have . . . 8

**7 Four routes, roughly in order of how well they fit you** **8**
7.1 Route 1: a German founder grant (best fit) . . . 8
7.2 Route 2: sell the analysis before selling the tool . . . 9
7.3 Route 3: design partners, not customers . . . 9
7.4 Route 4: build it inside a company . . . 9

**8 Who might buy this, and what to say** **9**
8.1 The positioning that is not crowded . . . 9
8.2 Why certification markets are the right target . . . 9
8.3 Where these people actually are . . . 9
8.4 What BoWatt appears to have done . . . 10

**9 Roadmap, if you decide to pursue it** **10**

**10 Risks worth saying out loud** **10**

# 1. The short version

You have two pieces of software. They do different jobs and it matters that they stay separate.

| | **GoalShift** | **ReqShift** |
|---|---|---|
| What you give it | A goal model you drew | A requirements document |
| What it gives back | Behavior classes, a statechart | A goal model, plus a report |
| Who does the thinking | A SAT solver | AI, then GoalShift |
| Can it be wrong? | No. Within your model it is proved | Yes. The extracted model is a draft |
| Needs internet | No | Yes |
| In your thesis and paper | **Yes** | **No** |
| Size | 445 lines of Python, 1,923 lines of HTML | about 3,300 lines across 11 files |

Think of a maths word problem. **GoalShift** is the calculator: give it the equation and it gives the exact answer, but it cannot read the word problem. **ReqShift** is the part that reads the word problem and writes down the equation, but it cannot do the arithmetic.

**Why they must stay separate. GoalShift** is the artifact your thesis describes and your paper will describe. Its SAT encoding *is* the contribution. If you change it, the numbers in your own chapter stop being reproducible. **ReqShift** imports **GoalShift** read only and never modifies it. Delete **ReqShift** tomorrow and **GoalShift** is exactly as it was.

# 2. How GoalShift works

## 2.1 What goes in

A *restricted type-level goal model*. In plain terms, a description of who is in the system, what each one is trying to achieve, and who needs help from whom.

- **Actor types** with a cardinality range. Not individual actors. "Up to five ambulances" is one actor type with an upper bound of five.
- **Goals** belonging to exactly one actor, one of them marked as that actor's root goal.
- **Decomposition** breaking a goal into sub-goals of the same actor, using AND, inclusive OR, or XOR.
- **Dependencies**, the only link allowed between different actors. A dependency carries a multiplicity: `1..*` means at least one present provider must supply it, `*..*` means every present provider must.

## 2.2 The four steps

1. **Encode to CNF.** Every actor instance and every goal instance becomes a Boolean variable. Four groups of clauses are generated: cardinality (including symmetry breaking so present instances form a prefix), decomposition, dependency, and linking (an absent actor satisfies nothing). This formula is $\Phi_{TGM}$.
2. **Walk the configurations.** A configuration is how many of each actor type are present. For each one, small SAT checks read off which actors can meet their root goal.
3. **Group into behavior classes.** Configurations with the same satisfaction pattern collapse into one class. These are equivalence classes, so testing one member tests them all.

4. **Derive the statechart.** Classes become states, and actors joining or leaving become transitions.

### 2.3 The scalability fix

This is worth recording because it changed what the tool can do.

The original engine answered "which behavior classes exist" by asking the solver to **list every complete satisfying assignment** and then grouping them. For the small running example that was 206 assignments, used to report 4 classes. The count roughly doubles with every added goal or actor, so it went from instant to never finishing with no warning in between.

| Dimension | Old ceiling | New ceiling |
|---|---|---|
| Actor types | 3 | 10 (7.5 s) |
| Instance actors per type | 2 | 30 (7.6 s) |
| Goals per actor | 3 | 8,191, at 49,152 variables (0.6 s) |

The replacement is a *projected* query. Only the actor-presence variables and the actor root goals are ever read, so the solver is asked about those directly and left to decide the rest internally. Goal tree size stops mattering, because a solver decides *whether* a big tree can be satisfied about as fast as a small one.

**The point for a defense or a paper.** The approach in the thesis was never the problem. The prototype's way of querying the solver was wasteful. Verified by running the old and new engines side by side on 29 generated models and comparing the full result, instance table, classes and statechart: **zero mismatches**.

### 2.4 The reference numbers

The Chapter 7 emergency coordination case study, which is the regression target:

| Engine | Variables | Clauses | Configurations | Solver calls | Classes |
|---|---|---|---|---|---|
| GoalShift (SAT) | 47 | 170 | 24 | 86 | 4 |
| z3 (SMT) | 15 | 22 | 24 | 12 | 4 |

## 3. How ReqShift works

### 3.1 The pipeline

| Stage | Who does it, and can it be wrong |
|---|---|
| Requirements document | your input |
| ↓ AI extraction | **a draft.** Review it |
| Goal model (GoalShift's own JSON) | |
| ↓ human review on the canvas | two minutes of your time |
| ↓ GoalShift and SAT | **proved** |
| Behavior classes, statechart, findings | |
| ↓ AI write-up (optional) | rewording of proved facts only |
| Report | |

The AI appears only at the two ends. The middle is your thesis and is untouched.

### 3.2 The files

| File | Lines | Job |
|---|---|---|
| `schema.py` | 224 | The JSON schema the AI must fill in, plus conversion to GoalShift's format |
| `extractor.py` | 263 | The only file that talks to a model. Two backends, one schema |
| `analyze.py` | 173 | Imports GoalShift, derives findings. **No AI involved** |
| `analyze_smt.py` | 632 | The experimental z3 backend. Opt in |
| `report.py` | 213 | Markdown report and the plain-text brief. **No AI involved** |
| `reqshift.py` | 197 | The command line tool |
| `app.py` | 282 | The web interface's server |
| `ui/index.html` | 964 | The whole interface, one self-contained file |
| `viewer.py` | 144 | Read-only browser for reports already on disk |

### 3.3 Why the AI cannot invent a defect

Every finding is computed arithmetically from the solver's output, inside `analyze.py`, which never calls a model at all. The optional write-up pass is given only the already-proved facts and told it may reword them but not add to them.

This is the whole design, and it is the answer to "how do I know your AI did not hallucinate this?"

### 3.4 What it finds

Four findings from the SAT path:

- **blocked**: an actor can be present and provably unable to meet its root goal. The expensive one, because the equipment is switched on and doing nothing.
- **never satisfied**: an actor reaches its root goal in no configuration at all. Usually an extraction mistake or an unachievable requirement.
- **initial degraded**: the minimum configuration, the state the system boots into, does not satisfy everything present.
- **inert cardinality**: a range the requirements allow that changes no outcome anywhere.

Two more become possible with the SMT path, and both are *solved for* rather than observed, so the numbers are exact:

- **capacity limit**: "Treatment Van is capped at 4 by Clinic Site capacity, not by its own bound of 8." The declared upper bound is not the real one.
- **deadline miss**: "cannot meet its 60 minute deadline, the soonest it can finish is 85."

### 3.5 Two ways to reach the AI

- **CLI backend**: uses the `claude` command from Claude Code. Needs no API key if you are already signed in. This is the zero-setup path and the one used for every result in this document.
- **SDK backend**: the official Anthropic SDK with structured outputs, which constrains the reply to the schema at the API level rather than by asking nicely. Needs an API key. Written from the documentation but **not yet tested**, because there is no key on this machine.

# 4. The SMT extension

## 4.1 Integer counts

`analyze_smt.py` analyses the same models with z3 integer variables instead of one Boolean per possible instance. **GoalShift** stays the default and stays untouched.

Two things follow. First, the lower bound may exceed one, so "at least three robots" becomes expressible where the SAT encoding rejects it outright. Second, and more interesting, a structural fact about the notation emerges:

> Every place a count is read, the only question asked is whether it is zero. A `1..*` dependency needs $n \geq 1$, a `*..*` dependency is vacuous at $n = 0$, and goals are false at $n = 0$. **Nothing in the notation can tell two present hospitals from three.** So the satisfaction pattern depends on the zero/nonzero vector alone, of which there are at most $2^k$ for $k$ actor types, however wide the ranges are.

That also *explains* the inert cardinality finding rather than merely detecting it, and it means the classes come back symbolic, which is a better answer than a list:

```
Class 4 (OK, OK, OK), 15 configurations
holds when n_D = 1 and 1 <= n_H <= 3 and 1 <= n_M <= 5
```

Four lines say everything a 24 cell grid says, and they stay four lines if the bounds go to a million. Measured: one million maximum instances per type gives $10^{18}$ configurations in 12 solver calls.

## 4.2 Timing and capacity

Four optional numeric fields, filled by the extractor only when the text states the figure: `duration` on a goal, `deadline` on an actor, `capacity` on an actor, `demand` on a dependency.

Adding any of them changes the shape of the problem. Once a real number is in play the answer stops depending only on whether a count is zero, so the fast path is dropped and configurations are walked. That is the honest cost.

What it buys is real. On the case study with each hospital admitting one patient, a **fifth behavior class appears** that cannot exist in the Boolean model: hospitals fine, ambulances present, ambulances blocked because there are not enough beds.

## 4.3 How it is kept honest

The SMT encoding rests on one assumption, that present instances of an actor type can be taken to behave alike when deciding that type's satisfaction. That should be exact here, since instances are interchangeable and share no constraint, but it is not proved.

So `bench/diff_smt.py` runs **GoalShift** as the oracle: **77 models compared, zero mismatches**, including the hand-built Chapter 7 model, four models extracted from real text, and 72 generated ones.

> A model analysed with the SMT engine is **no longer proved by the encoding the thesis describes**. The interface shows an amber banner saying so. Keep that label on anything you show anyone.

# 5. Evidence that it works

## 5.1 It finds real defects

`examples/welding_cell.txt` is a plausible robotic welding cell specification. Read it and nothing looks wrong.

From 349 words the extractor produced a model, and the solver proved that in **8 of 20 configurations** a piece of equipment is present and cannot do its job, including the configuration the cell boots into. The cause, traced back to the sentences responsible: two requirements set *upper* bounds on robots and vehicles and never a lower one, so zero is legal and everything downstream stalls.

It also flagged that the numbers four and three carry no behavioural consequence at all, which is worth knowing before someone plans capacity around them.

## 5.2 It reproduces a model you built by hand

Your Chapter 7 case study, from the 143 words of section 7.2 alone, which contains no goal model, no AND/OR/XOR, and no multiplicity notation.

Three independent runs, one from the command line and two through the interface, all produced the same four behavior classes with the same 1/3/5/15 membership split as your hand-built model. Compared by actor role rather than column position, the result is **identical**.

> **Worth putting in the paper.** The extraction consistently placed the `*..*` dependency on a different edge than you did, and invented a dispatch-side goal because your section 7.2 describes hospitals reporting to the dispatch center without giving the center a goal that needs it. Two structurally different type-level models, identical behavior classes, reproduced across runs. That is evidence the behavior-class abstraction is robust to how a modeller draws the dependency.

## 5.3 It reads numbers out of prose

`examples/dialysis_unit.txt` states its figures in ordinary sentences. From 314 words the extractor pulled all six durations converted to one unit, a 120 minute deadline, capacity four from "each clinic site has four treatment bays", and demand one from "each van occupies one bay".

The timing was checked by hand: $6 + 4 + 22 + 45 + 8 = 85$ minutes, which is exactly what z3 returned as the soonest possible finish. The latency chain composes correctly across the dependency.

# 6. From prototype to product: the honest starting position

Everything above is real and measured. What follows is opinion and options.

## 6.1 What you have

- A working analyzer with a validated formal foundation and a thesis behind it.
- An AI front end that removes the step that stops people using formal tools, namely building the model by hand.
- A differentiator that is genuinely uncommon. Plenty of tools do "AI reads requirements". Very few reason about *membership change*, and that is your actual contribution.
- Evidence, not claims. Differential tests, reproduced results, measured ceilings.

## 6.2 What you do not have

Being clear about this is more useful than optimism.

- **No product.** A prototype that a knowledgeable author can run is a long way from software a stranger can rely on.
- **No customer.** Nobody has paid, and nobody has used it on their own document without you present.
- **No company, and no clarity on who owns the work.** See below.
- **Real notation limits.** No soft goals, no contribution links, lower bounds restricted in the SAT path, XOR restricted to two children, and sequencing lost to AND.
- **The extraction is a draft.** This is by design and it is the right design, but it means the tool assists an expert rather than replacing one.

**Sort the intellectual property before anything else.** The work grew out of a thesis at TH Bingen, supervised by Prof. Bandyszak, and a paper is being written jointly. Rules on student and university IP vary by institution and by how the work was funded. Ask the university's technology transfer office in writing, and agree with your advisor what is shared and what is yours, *before* you show it to a company. This is not legal advice and you should get some.

# 7. Four routes, roughly in order of how well they fit you

## 7.1 Route 1: a German founder grant (best fit)

The **EXIST-Gründerstipendium** is a federal programme for founders coming out of universities. It typically provides a monthly stipend for graduates for twelve months, plus money for materials and coaching, and requires a university to host the project.

Why it fits you specifically: a fresh master's thesis with a commercialisable result, an advisor who could host it, and a technology that came out of academic work. That is close to the profile the programme exists for.

What to do: talk to the TH Bingen transfer office and to Prof. Bandyszak about hosting. **Verify the current terms yourself**, since amounts and rules change and I cannot confirm today's figures. There is also **EXIST-Forschungstransfer** for research-heavy projects, which is larger and harder to get.

### 7.2 Route 2: sell the analysis before selling the tool

The fastest way to learn whether anyone wants this is to do the work by hand for money.

Offer a fixed price review: they send one real specification, you run the tool, you review the extracted model yourself, and you hand back a report naming the defects and the sentences responsible. Perhaps five to fifteen thousand euro for one specification.

This is far easier to sell than software, because it is a service with a clear deliverable and no integration. It also tells you exactly what to build, since you will discover on the first document what the notation cannot express.

### 7.3 Route 3: design partners, not customers

Before charging anyone, find three companies willing to give you a real specification and two hours of an engineer's time, in exchange for a free analysis.

You are not selling. You are finding out whether the defects you surface are ones they care about. If an engineer says "we already knew that", the wedge is wrong. If they go quiet and then ask who else has seen it, the wedge is right.

### 7.4 Route 4: build it inside a company

Joining a company already selling into this space and building it there removes the funding problem entirely, at the cost of ownership.

BoWatt is the obvious example and they rejected you, but for a Full Stack Engineer role, not for this. That is worth separating. A message that says "you build agentic requirements engineering, I wrote a thesis on formally verifying multi-actor requirements models and here is a report my tool produced from a spec" is a completely different approach from a job application, and it can be sent to the founders directly.

## 8. Who might buy this, and what to say

### 8.1 The positioning that is not crowded

Do not say "AI for requirements". That market is loud and you will be compared to better-funded companies.

Say instead:

> *We prove which configurations of your system still work when parts of it are missing or unavailable, and we cite the requirement sentence that causes each failure.*

Two claims in there are hard for competitors to match. *Prove* is real, because a solver produced it. *Cite the sentence* is real, because traceability is built in.

### 8.2 Why certification markets are the right target

Cars, planes, medical devices and industrial safety all live under standards that demand traceability and evidence: ISO 26262, DO-178C, IEC 61508. In those markets, formal evidence is not a nice-to-have, it is something they already pay people to produce.

Your test reduction argument lands here too. Behavior classes are equivalence classes, so a handful of scenarios provably cover every configuration. A validation engineer understands the value of that immediately.

### 8.3 Where these people actually are

- **Fraunhofer IESE, Kaiserslautern.** Does requirements engineering research directly with industry, in Germany, near you. Probably the single most useful first conversation available to

you, because they know both the research and the companies.

- **Conferences with industry tracks:** REFSQ (requirements engineering, European, industry-friendly), IEEE RE, MODELS, INCOSE symposiums, and ERTS for embedded and real-time systems.
- **Tool vendors as partners rather than competitors:** IBM DOORS, Siemens Polarion, PTC codebeamer, Jama Connect. These own the workflow already. Being an analysis that plugs into their ecosystem is a much shorter path than replacing them.
- **Your advisor's network.** Prof. Bandyszak's own MODELSWARD paper on type and instance goal models came from somewhere, and that somewhere has industrial contacts.

### 8.4 What BoWatt appears to have done

From public information only. They bootstrapped to over half a million euro of annual recurring revenue *before* raising their first round, and only then took money from tier-one investors and angels.

The lesson worth copying is the order. They sold something to somebody first. Revenue before funding is also the version of this that does not require you to convince an investor of anything.

## 9. Roadmap, if you decide to pursue it

Ordered so that each step is useful even if you stop after it.

1. **Publish the paper with Torsten.** Credibility first, and it costs you nothing extra since it is already in progress.
2. **Settle the IP question.** In writing.
3. **Run it on three real specifications** from three different companies, free. Learn what breaks.
4. **Fix the notation gaps those documents expose.** Almost certainly soft goals and better handling of ordering.
5. **Make the extraction reviewable by someone who is not you.** The canvas review step is the product, and right now it needs a console paste.
6. **Charge for one analysis.** The first invoice tells you more than any amount of planning.
7. **Only then** consider whether it is a company, a grant application, a licence to a tool vendor, or a job at a company that already sells here.

## 10. Risks worth saying out loud

- **Enterprise sales cycles in safety-critical industries run six to eighteen months.** Plan funding accordingly, which is another argument for the grant route.
- **Domain credibility matters more than technology** in these markets, and a recent graduate has little of it. A partner, an advisor, or an institute affiliation compensates.
- **The incumbents own the workflow.** Nobody rips out DOORS. Integration beats replacement.
- **Your notation restrictions are real** and the first industrial document you try will expose several at once. That is not failure, it is the roadmap.
- **The tool cannot tell a defect from a design decision.** In the welding cell, a degraded boot configuration was a real bug. In your case study, the identical finding is the intended result. Same output, opposite meaning. Say that before anyone else does.

**The honest summary.** You built the hard half. Anyone can bolt an AI onto the front of a tool. Almost nobody has the thing behind it that can prove the answer. What you do not yet have is a customer, and everything above is really just different ways of going and finding one.